Project Report: Question 5

Sayan Bhadra (BMAT 2429)
Subhojit Maji (BMAT 2434)

October 26, 2026

1 Problem Definition

We model the company as a rooted tree $T = (V, E)$. The CEO is the root. Every other employee has exactly one manager. A direct report is an employee whose manager is a given person. Each manager–employee link counts as one edge. We write $\mathrm{dist}_T(u, v)$ for the number of edges on the path between employees u and v.

We want the smallest team $S \subseteq V$ such that everyone is within distance two of a team member:

$$\min |S| \quad \text{such that} \quad \forall u \in V,\ \exists v \in S : \mathrm{dist}_T(u, v) \leq 2.$$

Employees who share a manager are called co-workers. Two different co-workers are always exactly two edges apart.

A team member covers four groups of people:

- the member themselves;
- their manager and their manager's manager;
- their direct reports and the direct reports of those reports;
- their co-workers.

2 Dynamic Programming States

For an employee u, let T_u be u together with everyone who reports to u, directly or through others. Below, a *parent* is an employee's manager, a *child* is a direct report, and a *grandparent* is a manager's manager.

A team member outside T_u can cover u only if they are u's parent, u's grandparent, or a co-worker. So the table only needs to know whether those three people are selected.

For each u, we build a table with entries $D_u(a, b, s, x, t)$. Each flag is 0 or 1:

- $a = 1$ means u's parent is selected.

- $b = 1$ means u's grandparent is selected.
- $s = 1$ means at least one co-worker of u is selected.
- $x = 1$ means u is selected.
- $t = 1$ means u or one of u's children is selected.

The entry D_u gives the smallest team inside T_u that matches these flags and covers everyone in T_u. If no such team exists, the entry is $+\infty$.

The flag t matters because u's parent can be covered by u or by one of u's children. The parent needs to know which of these holds.

3 Recurrence

Let $R(u)$ be the set of children of u. For each child c, let $x_c = 1$ if c is selected. Let $t_c = 1$ if c or one of c's children is selected. The parent flag for c is x, and the grandparent flag for c is a.

The recurrence is

$$D_u(a, b, s, x, t) = x + \min \sum_{c \in R(u)} D_c(x, a, s_c, x_c, t_c),$$

$$s_c = \bigvee_{d \in R(u),\, d \neq c} x_d,$$

$$t = x \vee \bigvee_{c \in R(u)} x_c.$$

The minimum only considers child choices that satisfy these equations and the coverage rule

$$x \vee a \vee b \vee s \vee \bigvee_{c \in R(u)} t_c = 1.$$

This rule checks every team member who could cover u. The terms x, a, b, and s cover u, its parent, its grandparent, and its co-workers. Each t_c covers u through a child or a grandchild. The cost counts x for u itself, plus the cost of each child's subtree.

4 Fast Evaluation

Trying every subset of children would take too long. The co-worker flag s_c for each child depends on whether any other child is selected. So we split the work into three cases based on how many children are selected.

Case 1: No child is selected. Every child has co-worker flag 0. We run a two-state table that tracks whether any t_c equals 1. If none of u, its parent, its grandparent, or a co-worker is selected, some t_c must be 1.

Case 2: Exactly one child j is selected. Child j has co-worker flag 0. Every other child has co-worker flag 1. A prefix sum adds up the costs before j, and a suffix sum adds up the costs after j. Together they give the cost for j without rescanning the children each time.

Case 3: Two or more children are selected. Every child has co-worker flag 1. A three-state table counts selected children as 0, 1, or at least 2.

Every possible selection falls into exactly one of these cases. So the algorithm never has to try every subset.

5 Pseudocode

The algorithm handles each manager after all of their children. For every table entry, it saves the choices that give the minimum. Those saved choices let us rebuild the team at the end.

```
Minimum-Team(manager[0..n-1])
    Build a list of children for each employee.
    Build an order that starts at the CEO and lists each manager before their reports.

    For each employee u, in reverse order:
        For each flag tuple (a,b,s,x,t):
            If x=1 and t=0, skip this state.
            best = infinity

            If t=x:
                Evaluate Case 1 with a two-state table over the values t_c.
                Require some t_c=1 only if nothing else covers u.
                Save the least cost and its choices.

            If t=1:
                Evaluate Case 2 with prefix and suffix sums.
                Save the least cost and its choices.
                Evaluate Case 3 with a count table over states 0, 1, 2+.
                Save the least cost and its choices.

            If best is finite, set D_u = x + best.
            Save the choices that give D_u.

    At the CEO, set a=b=s=0.
    Pick the least finite value over x and t.
    Follow the saved choices to return the team.
```

6 Correctness

Lemma. For every employee u and every flag tuple, the entry D_u equals the smallest valid team inside T_u.

Proof. We use induction on the height of T_u, which is the longest downward path from u.

If u has no children, then $t = x$ must hold. The state is feasible exactly when at least one of x, a, b, or s is 1. In that case the cost is x. So the lemma holds for leaves.

Now assume the lemma holds for every child of u. Take any valid team inside T_u. It sets each child's flags. The parent flag for each child is x. The grandparent flag is a. The co-worker flag is

s_c. The recurrence covers all of these choices. The subtrees of different children do not overlap, so the team size is x plus the sizes of the child teams.

In the other direction, take feasible child states that satisfy the recurrence. Their teams cover their own subtrees, and the coverage rule covers u. Joining these teams gives a valid team inside T_u. So the recurrence finds the smallest valid team. □

Theorem. The algorithm returns a minimum-size team for the whole company.

Proof. The CEO has no parent, no grandparent, and no co-workers. So $a = b = s = 0$ at the root. Every valid team for the company appears in one of the root entries. By the lemma, the smallest finite root entry is the best possible size. The saved choices rebuild a team of that size. □

7 Running Time and Space

Each employee has 32 table entries. An employee with d children takes $O(d)$ time per entry, because each of the three cases loops over the children once. The total work is

$$O\left(\sum_{u \in V} (1 + |R(u)|)\right) = O(n),$$

since the tree has $n - 1$ manager–employee links. Recovering the team also takes $O(n)$ time.

The algorithm stores one table per employee and one saved choice per entry, so it uses $O(n)$ space.

Any correct method must read every employee record at least once, which takes $\Omega(n)$ time. So $O(n)$ is the best possible worst-case bound. The algorithm is deterministic, so this bound holds for every input of size n.

8 Examples

Employees are numbered from 0 in the manager list. The value `-1` marks the CEO.

For `[-1, 0, 0, 1, 1, 2]`, employee 0 is the CEO. Employees 1 and 2 report to employee 0. Employees 3 and 4 report to employee 1. Employee 5 reports to employee 2. The CEO is within two edges of everyone, so the smallest team is `[0]`. The program prints `[1]` because it shows employee numbers starting at 1.

For `[-1, 0, 1, 2, 3, 4, 5]`, the company is a chain of seven employees. One team member can cover at most five people in a chain, so a single member is not enough. Two members can cover all seven. One valid answer is zero-based indices `[2, 6]`, which are employees 3 and 7 when numbered from 1. Other teams of size two also work.

A Python Implementation

The variable `children` holds each employee's direct reports. The flag `parent_flag` tells whether the manager is selected. The flag `grandparent_flag` tells whether the manager's manager is selected. The flag `co_worker_flag` tells whether a co-worker is selected. The code saves the best choice for each entry, so the team can be rebuilt at the end.

```
from itertools import product
from math import inf


def minimum_team(manager):
    size = len(manager)
    if size == 0:
        return []

    roots = [employee_id for employee_id, boss in enumerate(manager) if boss == -1]
    if len(roots) != 1:
        raise ValueError("Expected exactly one CEO")
    children = [[] for _ in manager]
    for employee, boss in enumerate(manager):
        if boss == -1:
            continue
        if boss < 0 or boss >= size or boss == employee:
            raise ValueError("Invalid manager index")
        children[boss].append(employee)

    root = roots[0]
    order = [root]
    for employee in order:
        order.extend(children[employee])
    if len(order) != size:
        raise ValueError("The hierarchy is not a connected tree")

    def best_down(table, parent_flag, grandparent_flag,
                  co_worker_flag, root_flag):
        value0 = table[parent_flag, grandparent_flag,
                       co_worker_flag, root_flag, 0]
        value1 = table[parent_flag, grandparent_flag,
                       co_worker_flag, root_flag, 1]
        return (value0, 0) if value0 <= value1 else (value1, 1)

    tables = [None] * size
    decisions = [None] * size

    for employee in reversed(order):
        child_list = children[employee]
        table = {state: inf for state in product((0, 1), repeat=5)}
        saved = {}

        for state in product((0, 1), repeat=5):
            parent_flag, grandparent_flag, co_worker_flag, root_flag, down_flag = state
            if root_flag and not down_flag:
                continue
            best_cost, best_plan = inf, None

            if down_flag == root_flag:
                costs = [0, inf]
                back = []
                for child in child_list:
                    next_costs = [inf, inf]
                    previous = [None, None]
                    for old_or in (0, 1):
                        for child_down in (0, 1):
                            new_or = old_or | child_down
```

```
                value = tables[child][root_flag, parent_flag,
                                      0, 0, child_down]
                candidate = costs[old_or] + value
                if candidate < next_costs[new_or]:
                    next_costs[new_or] = candidate
                    previous[new_or] = (old_or, child_down)
        costs, back = next_costs, back + [previous]

    valid_or = (0, 1) if (root_flag or parent_flag or
                          grandparent_flag or co_worker_flag) else (1,)
    final_or = min(valid_or, key=lambda flag: costs[flag])
    candidate = costs[final_or]
    if candidate < inf:
        child_flags = [0] * len(child_list)
        current_or = final_or
        for index in range(len(child_list) - 1, -1, -1):
            current_or, child_flags[index] = back[index][current_or]
        best_cost = candidate
        best_plan = [(0, 0, flag) for flag in child_flags]

if down_flag:
    unselected_costs, unselected_down_flags, selected_costs = [], [], []
    for child in child_list:
        child_table = tables[child]
        value, flag = best_down(child_table, root_flag,
                                parent_flag, 1, 0)
        unselected_costs.append(value)
        unselected_down_flags.append(flag)
        selected_costs.append(
            child_table[root_flag, parent_flag, 0, 1, 1]
        )

    prefix = [0]
    for value in unselected_costs:
        prefix.append(prefix[-1] + value)
    suffix = [0] * (len(child_list) + 1)
    for index in range(len(child_list) - 1, -1, -1):
        suffix[index] = suffix[index + 1] + unselected_costs[index]

    one_cost, one_index = inf, None
    for index in range(len(child_list)):
        candidate = (selected_costs[index] + prefix[index] +
                     suffix[index + 1])
        if candidate < one_cost:
            one_cost, one_index = candidate, index
    if one_cost < best_cost:
        best_cost = one_cost
        best_plan = [
            (1, 0, 1) if other == one_index else
            (0, 1, unselected_down_flags[other])
            for other in range(len(child_list))
        ]

    counts, back = [0, inf, inf], []
    for child in child_list:
        child_table = tables[child]
        free_cost, unselected_down_flag = best_down(
            child_table, root_flag, parent_flag, 1, 0
        )
        options = [
            (0, unselected_down_flag, free_cost),
            (1, 1, child_table[root_flag, parent_flag, 1, 1, 1])
        ]
        next_counts = [inf, inf, inf]
        previous = [None, None, None]
        for old_count in range(3):
            for child_flag, child_down, value in options:
                new_count = min(2, old_count + child_flag)
```

```
                            candidate = counts[old_count] + value
                            if candidate < next_counts[new_count]:
                                next_counts[new_count] = candidate
                                previous[new_count] = (
                                    old_count, child_flag, child_down
                                )
                        counts, back = next_counts, back + [previous]

                    if counts[2] < best_cost:
                        plan = [(0, 1, 0)] * len(child_list)
                        count = 2
                        for index in range(len(child_list) - 1, -1, -1):
                            count, child_flag, child_down = back[index][count]
                            plan[index] = (child_flag, 1, child_down)
                        best_cost, best_plan = counts[2], plan

                if best_cost < inf:
                    table[state] = root_flag + best_cost
                    saved[state] = best_plan

            tables[employee], decisions[employee] = table, saved

        _, root_flag, down_flag = min(
            (tables[root][0, 0, 0, root_flag, down_flag], root_flag, down_flag)
            for root_flag in (0, 1) for down_flag in (0, 1)
        )
        team, stack = [], [(root, 0, 0, 0, root_flag, down_flag)]
        while stack:
            employee, parent_flag, grandparent_flag, co_worker_flag, root_flag, down_flag = stack.pop()
            if root_flag:
                team.append(employee)
            plan = decisions[employee][
                parent_flag, grandparent_flag, co_worker_flag, root_flag, down_flag
            ]
            for child, (child_flag, child_co_worker, child_down) in zip(
                    children[employee], plan):
                stack.append((child, root_flag, parent_flag, child_co_worker,
                              child_flag, child_down))
        return team


if __name__ == "__main__":
    manager = [-1, 0, 0, 1, 1, 2]
    print("Selected employees:",
          [employee + 1 for employee in minimum_team(manager)])
```